



Deep Learning

Índice

Introducción a Deep Learning (1/2)

01

Redes Neuronales Convolucionales (CNN)

02

Redes Neuronales Recurrentes (RNN) y LSTM

03



Introducción



Deep Learning

Definición

Es un subconjunto de técnicas de ML que imita la estructura y el funcionamiento del cerebro humano. Adquiere conocimientos a partir de datos no estructurados y emplea algoritmos complejos para entrenar redes neuronales artificiales.

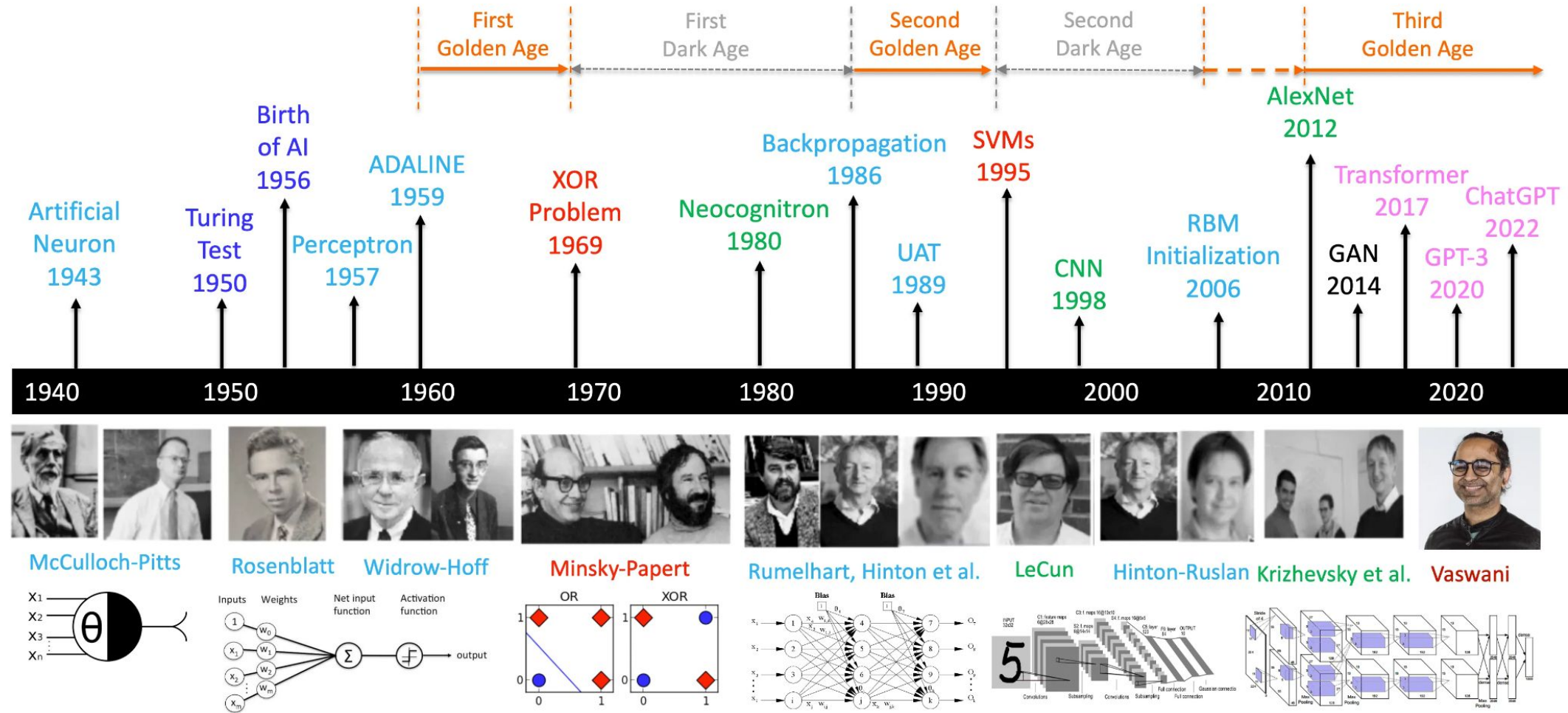
El concepto deep learning se introduce por primera vez en 2006 por Geoffrey Hinton (Google) aunque el concepto de red neuronal es de los años 50 del siglo XX.



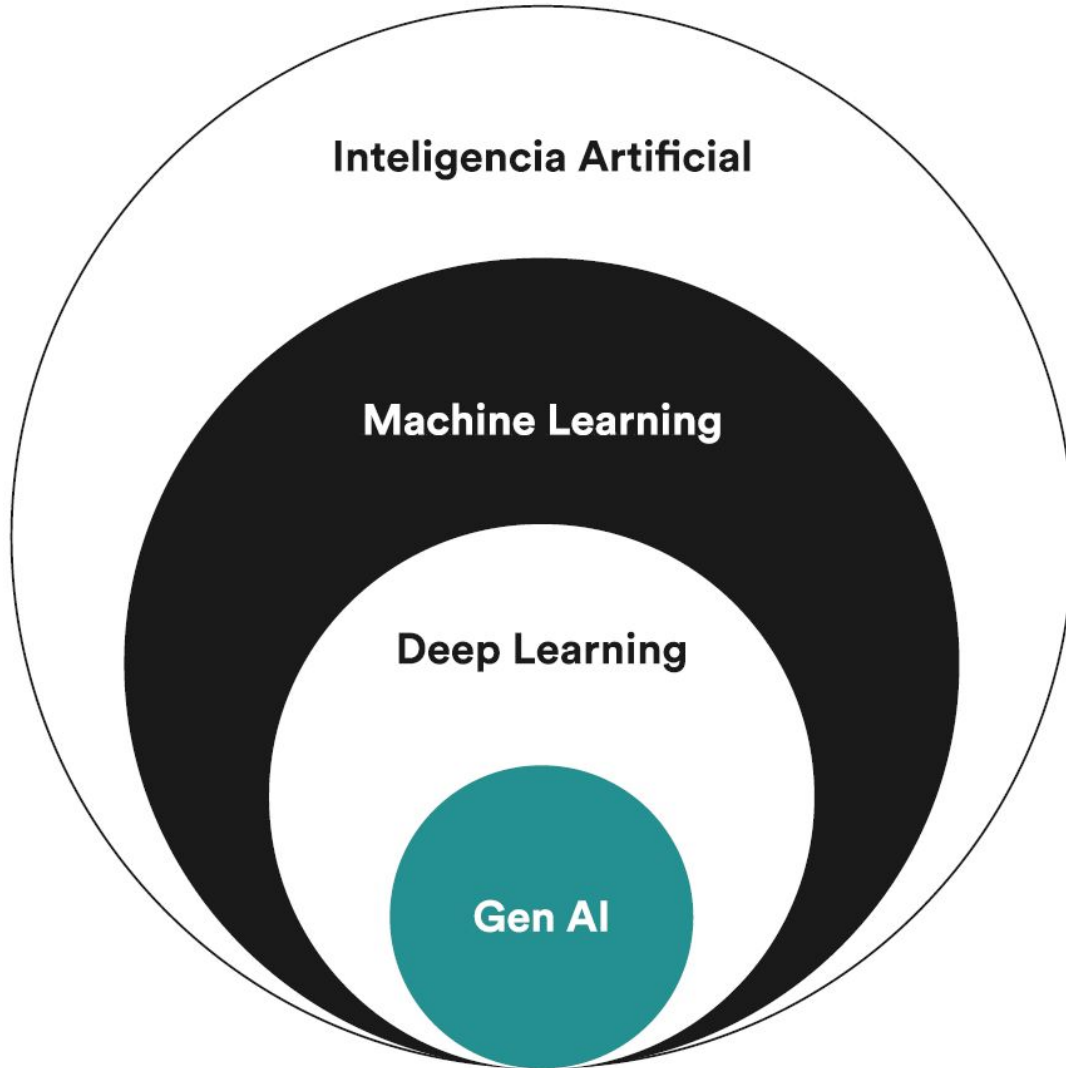
**Es capaz de tratar con datos de entrada más complejos:
no estructurados, audio, imágenes, video.**

Se puede emplear en los casos que hemos visto de clasificación y regresión. Y es especialmente útil en tareas de reconocimiento de imágenes, el procesamiento del lenguaje natural, la traducción automática, la conducción autónoma...

Historia



Deep Learning



1

Artificial Intelligence

A technique which enables machines to mimic human behaviour.

2

Machine Learning

Subset of AI techniques which use statistical methods to enable machines to improve with experience.

3

Deep Learning

Subset of ML which makes the computation of multi-layer neural networks feasible.

4

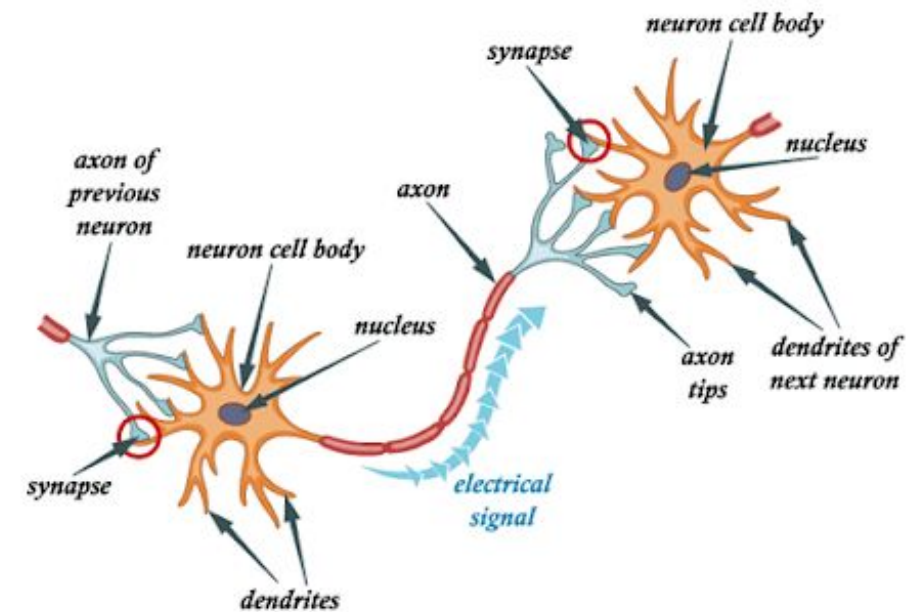
Generative AI

Generative Language Models
Generative Image Models



Neurona biológica (BNN)

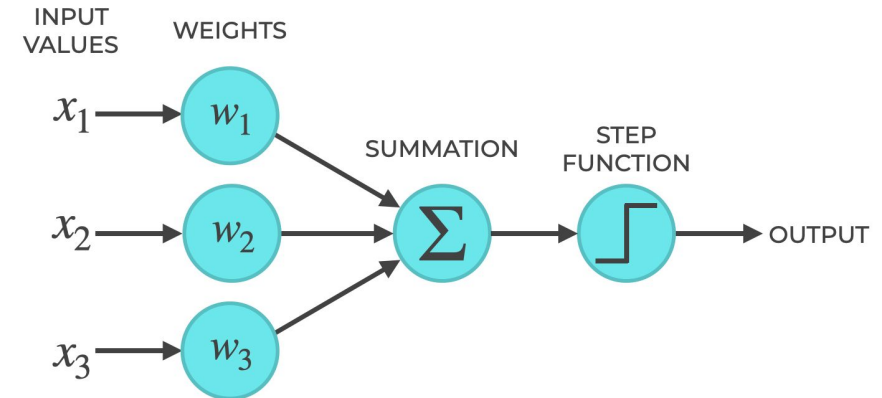
- Las neuronas artificiales (ANN) asemejan el comportamiento de las neuronas biológicas (BNN)
- Mediante la sinapsis, las neuronas conectan a través de los terminales del axón con las dendritas o núcleos de otras neuronas.
- Las neuronas reciben impulsos eléctricos (señales) de otras neuronas y esto hace que desencadenen su propia señal hacia otras neuronas.
- Las neuronas están organizadas en redes de modo que una neurona puede estar conectada con miles de otras neuronas





Perceptron

- El perceptrón es la arquitectura ANN más simple.
- También se le puede denominar Threshold logic Unit (TLU) o Linear Threshold Unit (LTU)
- $x_1, \dots, x_n$ son las entradas y cada entrada lleva asociado un peso $w_1, \dots, w_n$.
- A estos input ponderados por los pesos se les aplica una función step. En sentido genérico es la función de activación.



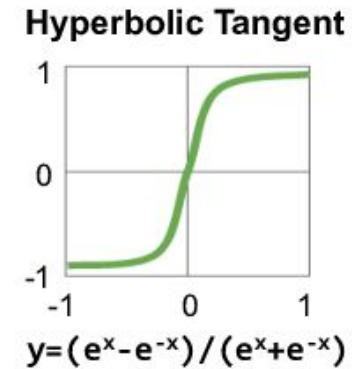
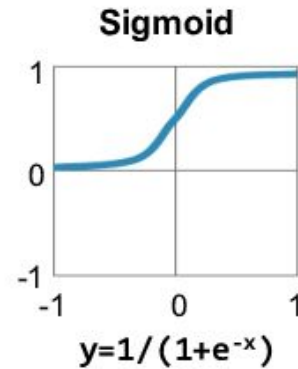
$$heaviside(z) = \begin{cases} 1 & \text{si } z \geq 0, \\ 0 & \text{si } z < 0. \end{cases}$$

$$sgn(z) = \begin{cases} -1 & \text{si } z < 0 \\ 0 & \text{si } z = 0 \\ 1 & \text{si } z > 0 \end{cases}$$

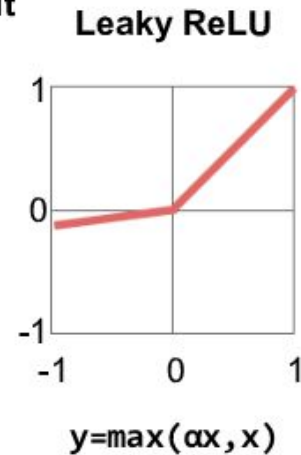
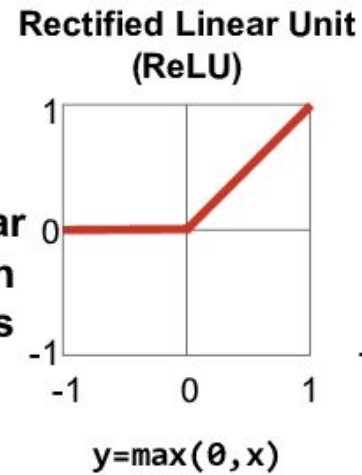
Funciones de activación



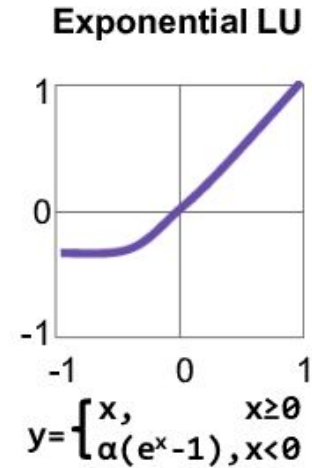
**Traditional
Non-Linear
Activation
Functions**



**Modern
Non-Linear
Activation
Functions**



α = small const. (e.g. 0.1)





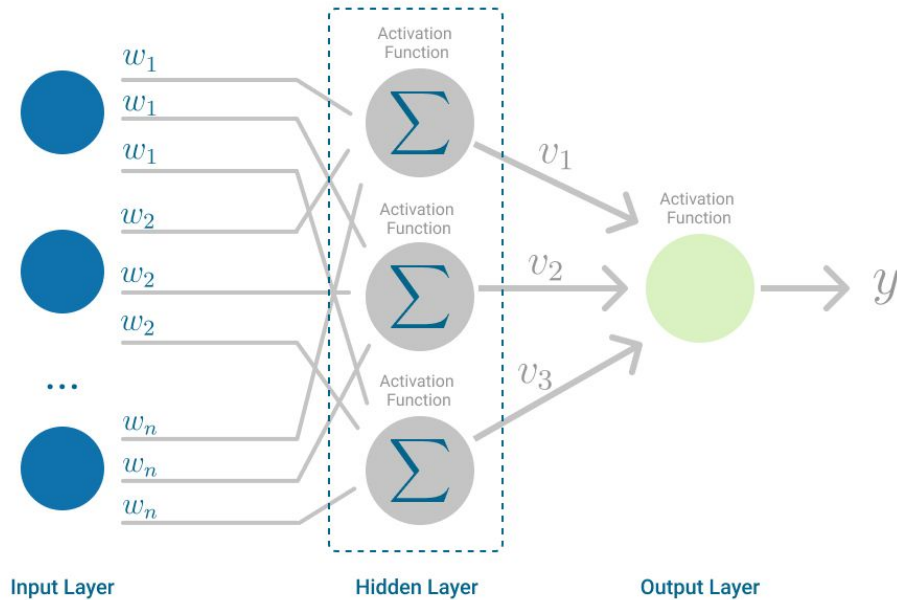
Teorema de Aproximación Universal (UAT)

El Teorema de Aproximación Universal afirma que una red neuronal con al menos una capa oculta y funciones de activación no lineales (como ReLU, sigmoide o tanh) puede aproximar cualquier función continua en un intervalo cerrado, con la precisión que se desee.

En otras palabras: una red suficientemente grande puede aprender cualquier patrón, sin importar lo complejo que sea, siempre que tenga los datos adecuados.

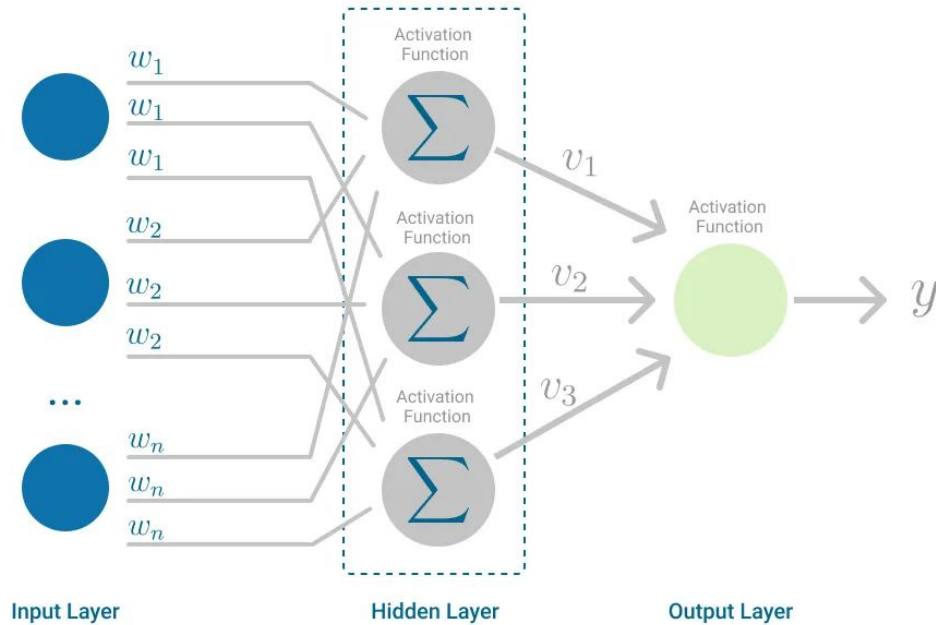
Este teorema es fundamental porque justifica matemáticamente por qué las redes neuronales son tan poderosas para tareas como reconocimiento de voz, visión por computadora o predicción de series temporales.

Perceptron Multicapa (MLP)



- 1 El perceptrón por sí solo no tiene una gran capacidad de resolución de problemas.
- 2 La adición de capas ocultas (hidden layer) permite que el perceptrón multicapa (MLP) capture y represente relaciones complejas y no lineales en los datos.
- 3 Estas capas ocultas mejoran significativamente la capacidad de aprendizaje de la red, permitiéndole resolver problemas de alta complejidad
- 4 El algoritmo que se emplea para estimar los parámetros del sistema es backpropagation

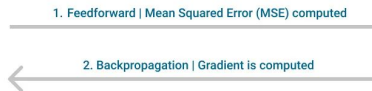
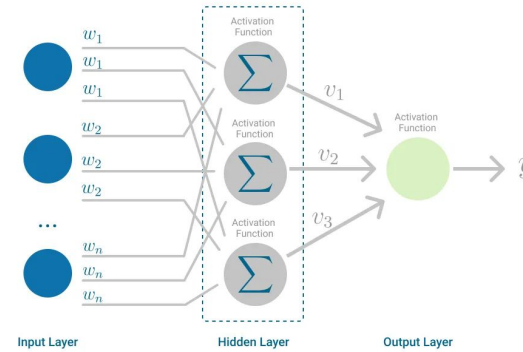
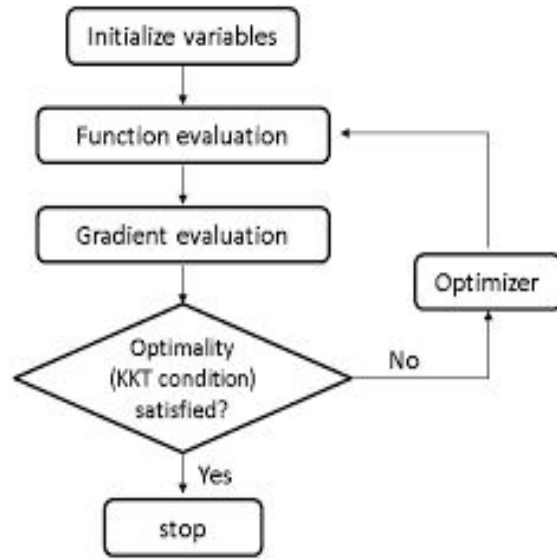
Estimación: Back Propagation



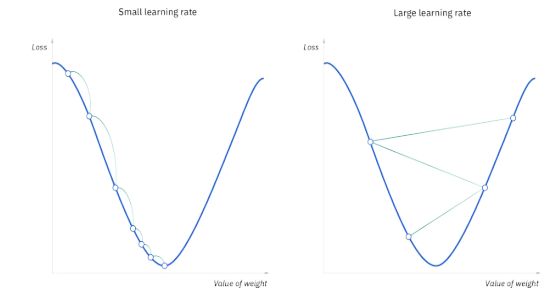
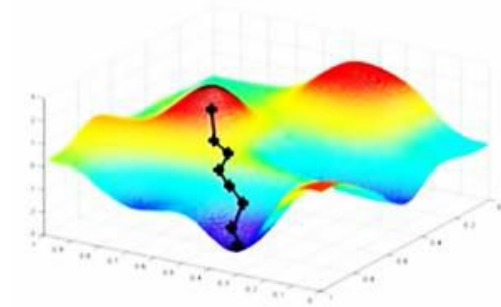
1. Feedforward | Mean Squared Error (MSE) computed

2. Backpropagation | Gradient is computed

- 1 Back propagation permite al perceptrón multicapa ajustar iterativamente los pesos (w_{ij}) en la red, con el objetivo de minimizar la función de coste.
- 2 Existe un requisito imprescindible para que backpropagation funcione correctamente. La función que combina entradas y pesos en una neurona, como la suma ponderada, y la función de activación, deben ser derivables.
- 3 Estas funciones deben tener una derivada acotada, ya que el descenso por gradiente es típicamente la función de optimización utilizada en el perceptrón multicapa.



$$\Delta_w(t) = -\underbrace{\varepsilon}_{\text{Bias}} \underbrace{\frac{dE}{dw(t)}}_{\text{Gradient Current Iteration}} + \underbrace{\alpha}_{\text{Learning Rate}} \underbrace{\Delta_w(t-1)}_{\text{Gradient Previous Iteration}}$$



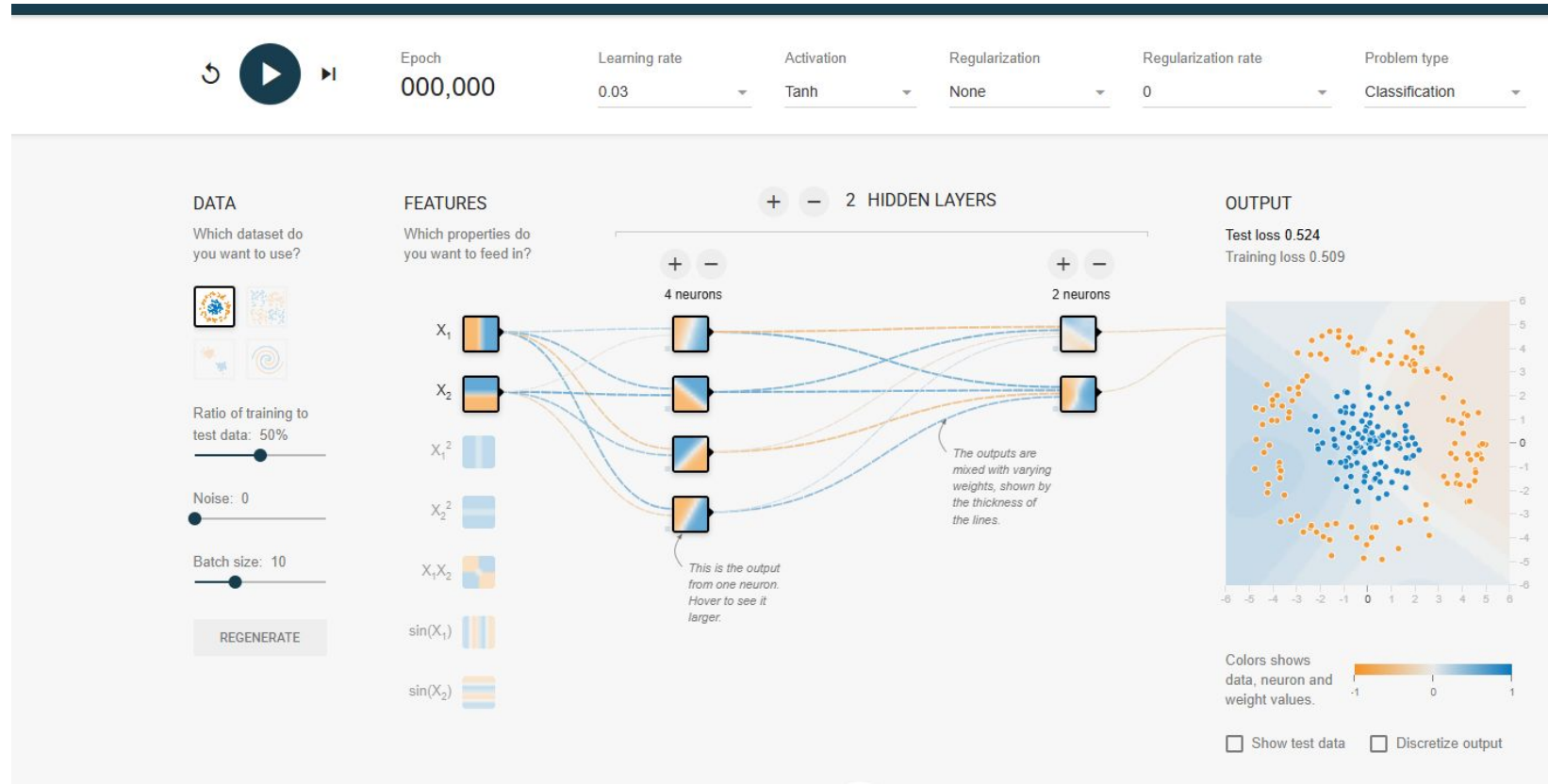
Estimación: Back Propagation

- 1 Las sumas ponderadas se transmiten a través de las capas y dan una primera estimación. (forward propagation)
- 2 Se calcula el error de la predicción (MSE).
- 3 Se va hacia atrás viendo la contribución al error de cada conexión.
- 4 Se modifican ligeramente los pesos (w_{ij}) con el fin de reducir el error mediante el método del gradiente

El **gradiente** es un vector que se calcula derivando la función de coste con respecto a los parámetros del modelo. Esto nos indica la dirección donde se reduce más este error.



Neural Network playground





Tensor Flow vs Pytorch



Facilidad de uso

- 1 **PyTorch** tiene una interfaz más intuitiva y fácil de usar, lo que facilita la escritura y depuración del código para los desarrolladores.
- 2 **TensorFlow** tiene una curva de aprendizaje más pronunciada, pero ofrece mayor flexibilidad y control sobre la arquitectura y el proceso de entrenamiento del modelo.

Grafos Computacionales Dinámicos vs. Estáticos

- 1 **TensorFlow** usa **grafos computacional estático**, lo que significa que el grafo se compila antes de la ejecución y no puede modificarse en tiempo de ejecución.
- 2 **PyTorch** usa un **grafos computacional dinámico**, permitiendo construir y modificar el grafo sobre la marcha, lo que lo hace más flexible y fácil de depurar.

Despliegue

- 1 **TensorFlow** tiene una infraestructura de despliegue más madura, lo que permite una integración sencilla con sistemas de producción.
- 2 **PyTorch** sigue siendo relativamente nuevo y carece de algunas de las funcionalidades avanzadas de despliegue que ofrece TensorFlow.

Resumen

TensorFlow es ideal para proyectos más grandes y complejos, donde el rendimiento y la escalabilidad son aspectos clave. PyTorch, por otro lado, es mejor para prototipado rápido y experimentación, donde la flexibilidad y la facilidad de uso son más importantes.



Redes neuronales con tensor flow



Capas de la red neuronal

Características

Flatten

```
(tf.keras.layers.Flatten())
```

Convierte una matriz multidimensional en un vector 1D.

Se usa cuando la entrada es una imagen (por ejemplo, en CNNs) y necesitas transformarla en una sola lista de valores para conectarla a una capa densa.

Uso común: Después de una capa convolucional para preparar los datos para la capa totalmente conectada.

Dense

```
(tf.keras.layers.Dense(64, activation='relu'))
```

Es una capa totalmente conectada donde cada neurona de una capa está conectada con todas las neuronas de la siguiente capa.

Se utiliza en redes neuronales profundas para procesar datos estructurados.

Algunos tipos de funciones de activación:

- Relu (es la función por defecto)
- Softmax genera probabilidades
- Sigmoid para salidas de clasificación binaria
- Linear

Dropout

```
(tf.keras.layers.Dropout(0.5))
```

Reduce el sobreajuste eliminando aleatoriamente un porcentaje de neuronas durante el entrenamiento.

Ayuda a evitar que el modelo se vuelva demasiado dependiente de ciertas conexiones.

Conv2D

```
(tf.keras.layers.Conv2D(32, (3,3), activation='relu'))
```

Se usa en redes neuronales convolucionales (CNNs) para procesar imágenes.

Aplica filtros a la imagen para extraer características relevantes como bordes, texturas, etc.

MaxPooling2D

```
(tf.keras.layers.MaxPooling2D(pool_size=(2,2)))
```

Reduce la dimensión espacial de la imagen seleccionando los valores máximos de una región.

Size 2,2 determina el tamaño de la ventana que se desliza sobre la imagen

BatchNormalization

```
(tf.keras.layers.BatchNormalization())
```

Normaliza los valores de activación en cada capa para estabilizar el entrenamiento y mejorar la convergencia.



Redes neuronales: ejemplo con tensor flow

```
# build the model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])

# compile the model
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

# train the model
model.fit(x_train, y_train, epochs=10)
```



Optimizadores

Descenso de Gradiente Estocástico (SGD)

Es el algoritmo de optimización más básico, que actualiza los pesos del modelo multiplicando la tasa de aprendizaje por la derivada de la función de pérdida. Variantes de SGD, como Momentum y Nesterov accelerated gradient, pueden mejorar la eficiencia en la optimización.

Adam (Adaptive Moment Estimation)

Es un método que calcula tasas de aprendizaje adaptativas para distintos parámetros. Es una combinación de AdaGrad y RMSProp, y es popular porque maneja bien gradientes dispersos en problemas ruidosos.

RMSprop (Root Mean Square Propagation)

Es un método adaptativo de tasa de aprendizaje no publicado, propuesto por Geoff Hinton en su curso. Es una extensión de AdaGrad, diseñada para manejar mejor los problemas de disminución rápida de la tasa de aprendizaje.



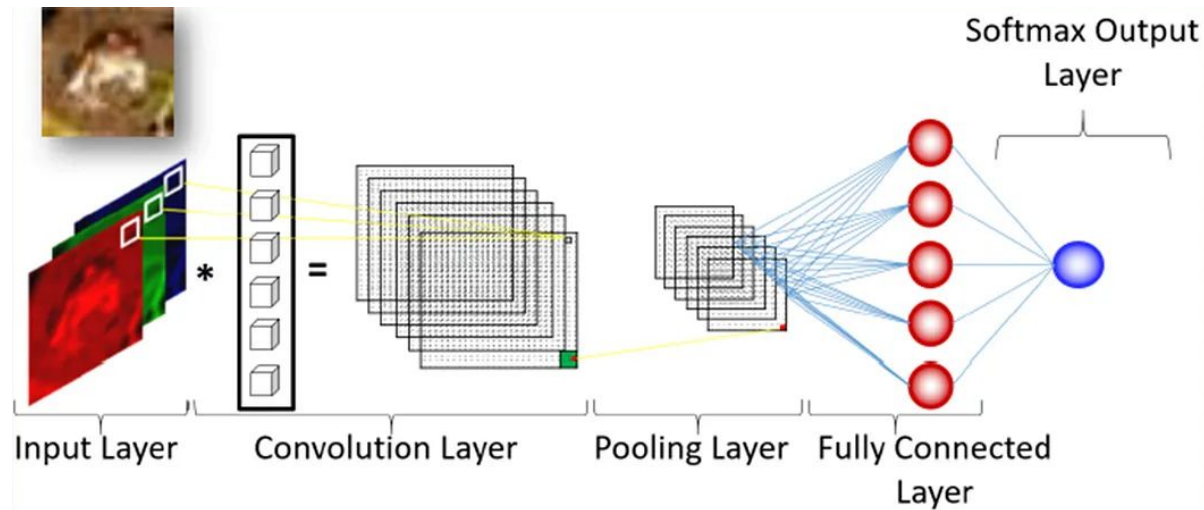
Redes Neuronales Convolucionales (CNN)



Redes Neuronales Convolucionales (CNN)

Una Red Neuronal Convolucional (CNN) es un tipo de red neuronal artificial diseñada para procesar y analizar datos con topologías en forma de cuadrícula, como imágenes y videos.

Podemos entender una CNN como un filtro multicapa que procesa imágenes para extraer características significativas y hacer predicciones.



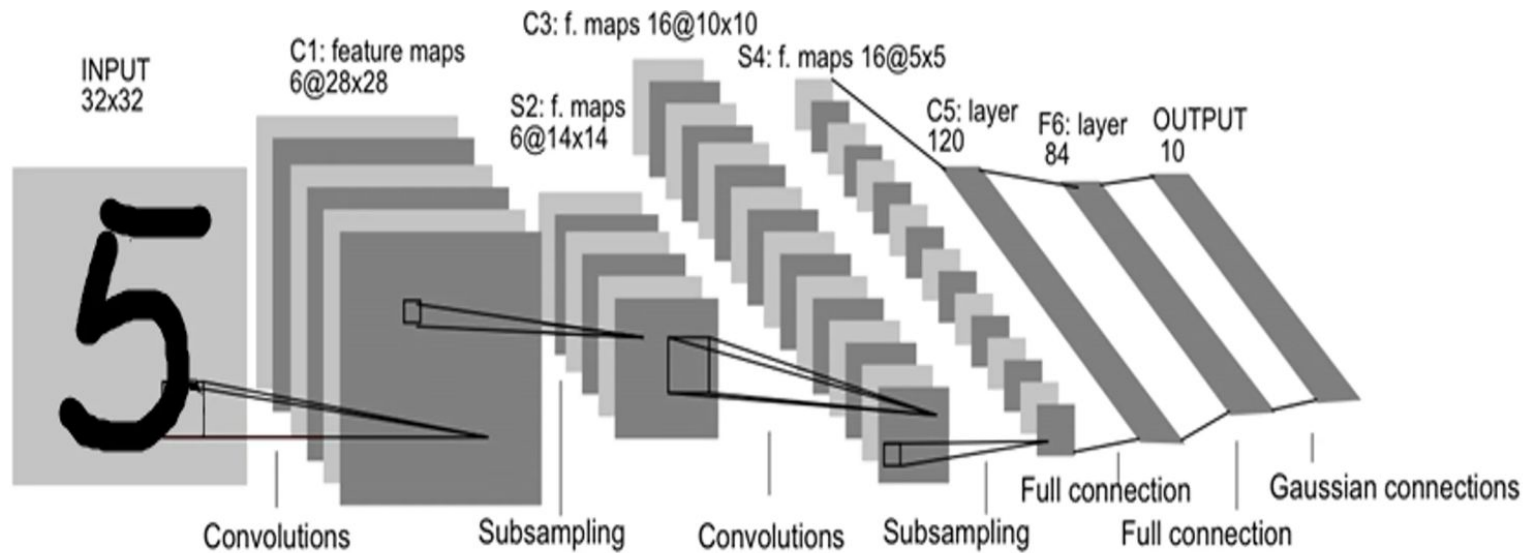
Una CNN funciona aplicando una serie de filtros sobre la imagen, extrayendo gradualmente características cada vez más complejas.

Los primeros filtros detectan características simples como bordes y líneas, mientras que los filtros posteriores identifican patrones más complejos, como formas y dígitos.



Redes Neuronales Convolucionales (CNN)

A finales de la década de 1980 y principios de la de 1990, Yann LeCun y su equipo evolucionan las redes CNN, introduciendo la arquitectura LeNet-5, diseñada específicamente para el reconocimiento de dígitos escritos a mano.



LeNet-5



Yann LeCun



Redes Neuronales Convolucionales (CNN)

Ejemplo con Tensor flow:

```
def define_model():  
    model = Sequential()  
    model.add(Conv2D(32, (3, 3), activation='relu',  
kernel_initializer='he_uniform',  
input_shape=(28, 28, 1)))  
    model.add(MaxPooling2D((2, 2)))  
    model.add(Flatten())  
    model.add(Dense(100, activation='relu',  
kernel_initializer='he_uniform'))  
    model.add(Dense(10, activation='softmax'))  
    # compile model  
    opt = SGD(learning_rate=0.01, momentum=0.9)  
    model.compile(optimizer=opt, loss='categorical_crossentropy',  
metrics=['accuracy'])  
    return model
```

————→ Capa convolucional para tratamiento de imágenes

————→ Reducción de dimensiones espaciales

————→ Convierte la matriz en un vector 1D

————→ Capa donde cada neurona está conectada con todas las capas de la neurona siguiente y softmax para que dé como salida probabilidades para las 10 categorías



CNN: otros algoritmos

AlexNet (2012)

Revolucionó el campo del aprendizaje profundo al ganar el concurso ImageNet con una arquitectura compuesta por múltiples capas convolucionales, max-pooling y capas completamente conectadas. Su diseño, con el uso de ReLU, dropout y procesamiento paralelo, permitió un entrenamiento más eficiente y preciso en la clasificación de imágenes

VGGNet (2014)

Desarrollada por el Visual Geometry Group de Oxford, VGGNet enfatizó arquitecturas más profundas con filtros de convolución más pequeños (3×3), logrando una precisión notable.

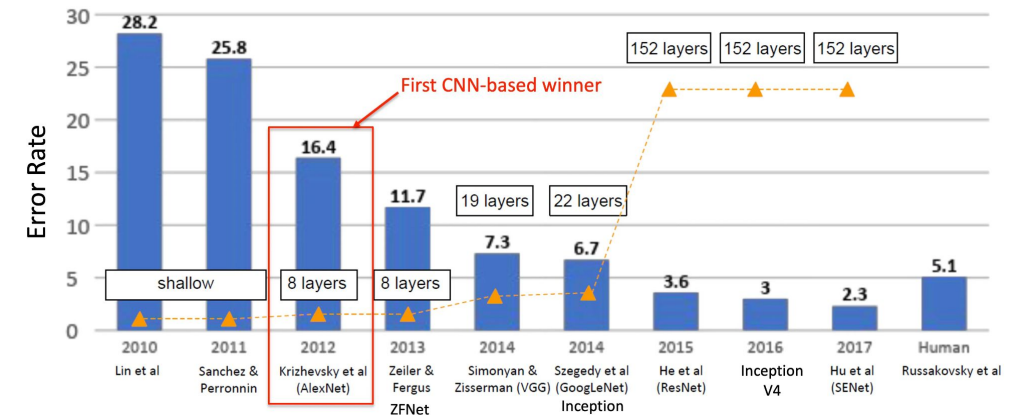
GoogLeNet/Inception (2014)

Introdujo los módulos Inception, que permitieron a la red capturar características a múltiples escalas de manera eficiente.

ResNet (2015)

Las Redes Residuales (ResNet) incorporaron conexiones de salto (skip connections), lo que permitió entrenar redes muy profundas mientras se mitigaba el problema del gradiente que se desvanece.

Winners of ImageNet Classification Challenge (ILSVRC)





CNN: Casos de uso

1

Visión por computadora

Las CNNs se han convertido en la base de la visión por computadora moderna, permitiendo avances en clasificación de imágenes, detección de objetos y segmentación semántica.

2

Imágenes médicas

Las CNNs se utilizan en tareas como diagnóstico de enfermedades, detección de tumores y cirugía asistida por imágenes, mejorando significativamente la precisión diagnóstica.

3

Vehículos autónomos

Las CNNs son fundamentales en los sistemas de percepción de los autos autónomos, permitiéndoles interpretar y responder a su entorno.



Redes Neuronales Convolucionales (CNN)

Las capas de una CNN pueden dividirse en tres tipos principales: capas convolucionales, capas de agrupamiento (pooling) y capas completamente conectadas.

Capas convolucionales

Estas capas aplican filtros, también llamados kernels, a la imagen. Cada filtro se mueve por la imagen y calcula valores que resaltan características específicas, como líneas horizontales, curvas o esquinas. El resultado es un mapa de características, que contiene información relevante para la tarea de clasificación o detección.

Capas de agrupamiento (Pooling Layers)

Estas capas realizan una operación de reducción de tamaño en los mapas de características, disminuyendo las dimensiones espaciales de los datos mientras conservan las características importantes. Esto ayuda a reducir la complejidad computacional y a prevenir el sobreajuste. El tipo más común de agrupamiento es el max pooling, que selecciona el valor máximo dentro de una pequeña vecindad de píxeles.

Capas completamente conectadas (Fully Connected Layers)

Son similares a las capas de las redes neuronales tradicionales. Conectan cada neurona de una capa con todas las neuronas de la siguiente. La salida de las capas convolucionales y de agrupamiento se aplanan y pasa por una o más capas completamente conectadas, lo que permite que la red haga una predicción final, por ejemplo reconocer el dígito en una imagen.



Redes Neuronales Recurrentes (RNN)

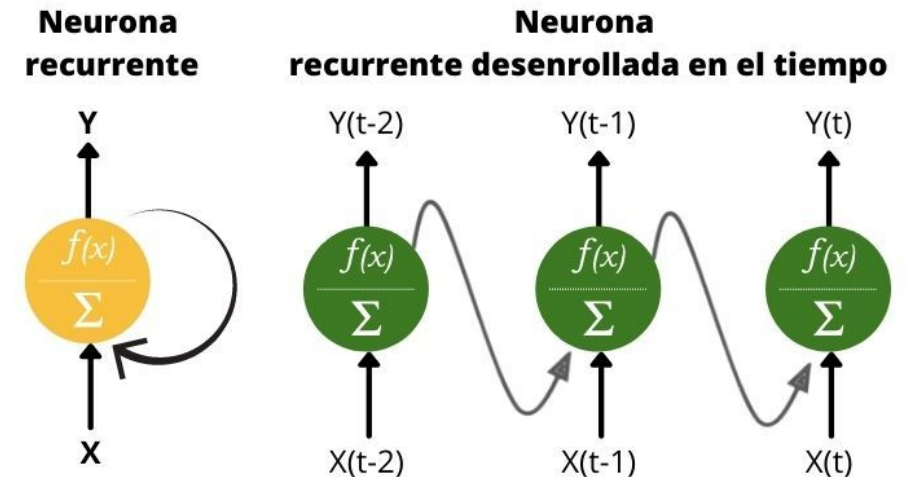


Redes Neuronales Recurrentes (RNN)

Las Redes Neuronales Recurrentes (RNNs) son un tipo de red neuronal artificial diseñada para procesar datos secuenciales, como series temporales, voz y lenguaje natural. Podemos imaginar una RNN como una cinta transportadora que procesa la información elemento por elemento, permitiéndole "recordar" información de elementos anteriores para hacer predicciones sobre el siguiente.

Un ejemplo puede ser que tenemos una secuencia de palabras y queremos que una computadora prediga la siguiente palabra en la secuencia. Una RNN funciona procesando cada palabra una a la vez y utilizando la información de las palabras anteriores para predecir la siguiente.

El componente clave de una RNN es la conexión recurrente, que permite que la información fluya de un paso de tiempo al siguiente. La conexión recurrente es un enlace dentro de una neurona que "recuerda" la información del paso de tiempo anterior.

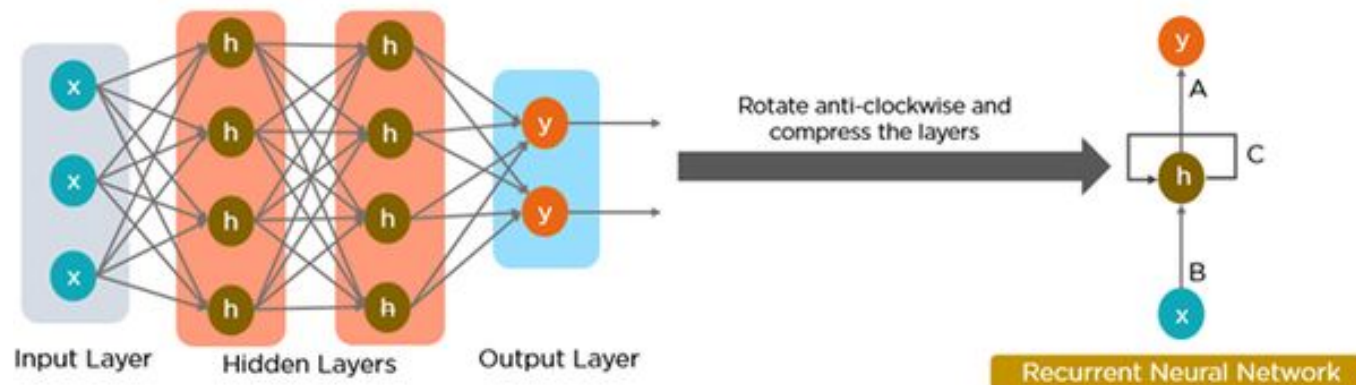




Redes Neuronales Recurrentes (RNN)

Una Red Neuronal Recurrente (RNN) se puede dividir en tres partes principales: la capa de entrada, la capa recurrente y la capa de salida.

- **Capa de entrada:** Recibe la información en cada paso de tiempo, como una palabra en una secuencia.
- **Capa recurrente:** Procesa la información de la capa de entrada, utilizando las conexiones recurrentes para "recordar" información de pasos de tiempo anteriores. Esta capa contiene un conjunto de neuronas, cada una con una conexión recurrente consigo misma y otra conexión con la entrada del paso de tiempo actual.
- **Capa de salida:** Genera una predicción basada en la información procesada por la capa recurrente. En el caso de generar la siguiente palabra en una secuencia, la capa de salida predeciría la palabra más probable que siga a las anteriores en la secuencia.





RNN: Otros algoritmos → LSTM

Long Short-Term Memory (LSTM): Permite recordar información a largo plazo en secuencias de datos, como texto, voz o series temporales. Funciona como una memoria que decide qué recordar y qué olvidar en cada paso de tiempo. Las LSTM hacen esto mediante tres "puertas":

- **Puerta de entrada:** Decide qué nueva información agregar a la memoria.
- **Puerta de olvido:** Elimina información innecesaria o irrelevante.
- **Puerta de salida:** Usa la información almacenada para hacer una predicción.

Y dos estados:

- **Estado de celda (Cell State)** Es como una memoria a largo plazo que almacena información relevante a través de los pasos de tiempo. Se mantiene y actualiza mediante las compuertas para recordar datos importantes y olvidar lo innecesario.
- **Estado oculto (Hidden State)** Actúa como la memoria a corto plazo, transportando información entre las unidades de la red en cada paso de tiempo. Es la parte visible de la red que se usa para tomar decisiones o hacer predicciones en cada instante.

Cada paso que da la red en la secuencia, se ajusta la memoria de manera inteligente para enfocarse solo en lo más importante.



RNN: Otros algoritmos → GRU

Kyunghyun Cho et al. introdujeron las GRUs (Gated Recurrent Units), una versión simplificada de las LSTMs que también emplea un mecanismo de Puertas para regular el flujo de información.

A diferencia de las LSTMs, que tienen tres puertas y dos estados, las GRUs funcionan con dos puertas y un solo estado.

Y dos estados:

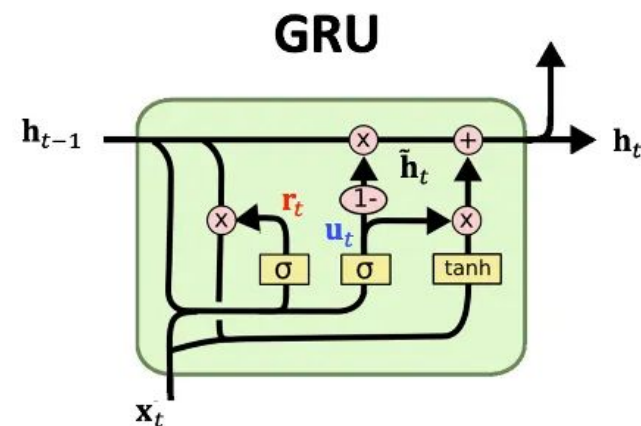
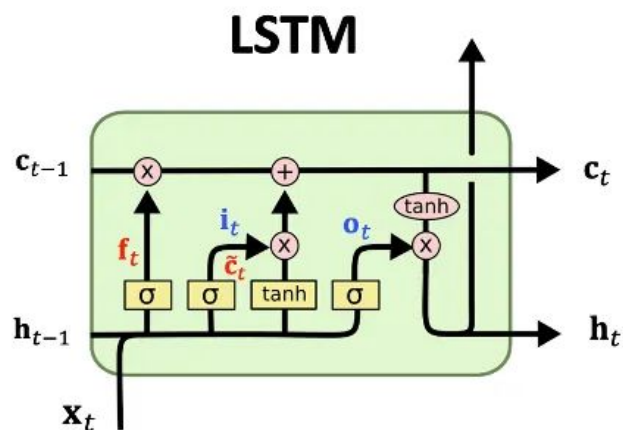
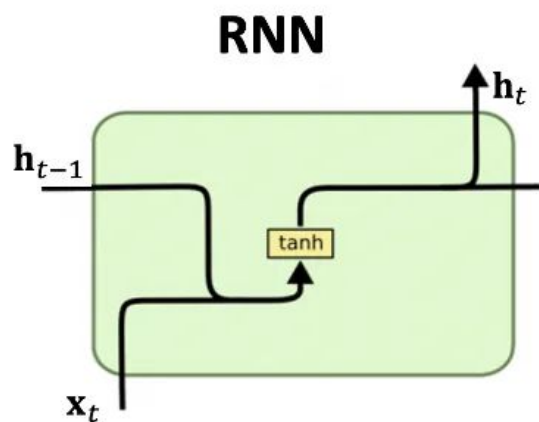
- Las Puertas de **Olvido y Entrada** de las LSTMs se combinan en una única compuerta de actualización (Update gate), que decide cuánta información pasada conservar y cuánta nueva información incorporar.
- La puerta de **Salida** de las LSTMs se reemplaza por la compuerta de reinicio (Reset gate) en las GRUs, que determina cuánta información pasada debe "reiniciarse" o olvidarse antes de integrar nueva información.

Las GRUs tienen menos parámetros que las LSTMs, lo que las hace generalmente más rápidas de entrenar.

RNN: Otros algoritmos → Seq2Seq



Ilya Sutskever y su equipo introdujeron el modelo Seq2Seq, que utiliza una arquitectura de codificador-decodificador para mapear secuencias de entrada en secuencias de salida. Este modelo ha sido ampliamente utilizado en tareas como traducción automática, reconocimiento de voz y resumen de texto.





RNN: casos de uso



Procesamiento del Lenguaje Natural (NLP)

Las RNNs han revolucionado el campo del procesamiento del lenguaje natural, permitiendo grandes avances en tareas como modelado del lenguaje, traducción automática, análisis de sentimiento y generación de texto.



Reconocimiento de voz

Las RNNs se utilizan ampliamente en sistemas de reconocimiento de voz, donde modelan las dependencias temporales del lenguaje hablado para convertir señales de audio en texto.



Predicción de series temporales

Las RNNs son efectivas para la predicción de series temporales, ya que modelan las dependencias temporales en datos secuenciales para prever valores futuros.



RNN: ejemplo en Python

```
def create_gru_model(input_length, vocab_size, embedding_dim=100,  
gru_units=128):
```

```
    # Create a sequential model (stacking layers on top of each other).  
    model = Sequential()
```

```
    # Create an embedding layer to represent words as vectors.  
    model.add(Embedding(vocab_size, embedding_dim))
```

—————→ Capa embedding para representar palabras como vector

```
    # Add a GRU layer (type of recurrent layer) to process sequences of words.  
    model.add(GRU(gru_units))
```

—————→ Capa GRU para procesar secuencias de palabras

```
    # Add a dense layer (fully connected) with sigmoid activation  
    # for binary classification (0 or 1).  
    model.add(Dense(1, activation='sigmoid'))
```

—————→ Capa densa con función sigmoidea: clasificación binaria

```
    # Compile the model with loss function, optimizer, and metrics.  
    model.compile(loss='binary_crossentropy', optimizer='adam',  
metrics=['accuracy'])
```

```
    return model
```



Otras arquitecturas: GAN

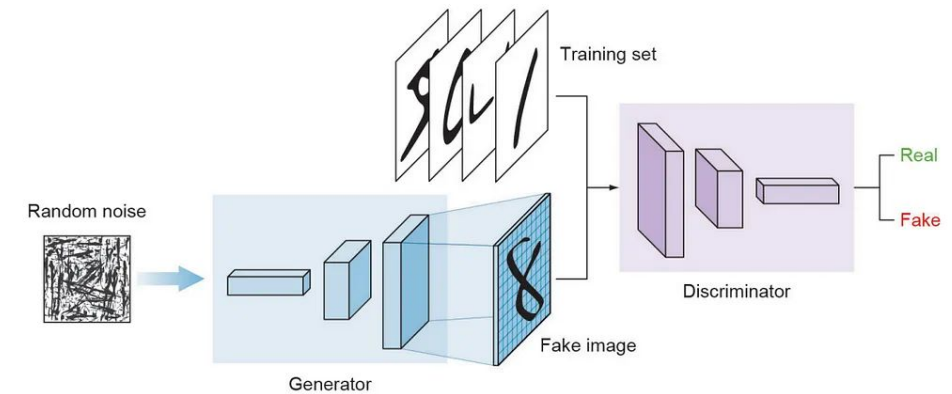
Son un tipo de arquitectura neuronal introducida por Ian Goodfellow y otros investigadores en junio de 2014.

Las **GANs** destacan por su capacidad para crear imágenes, videos, música y texto hiperrealistas.

Las **GANs** tienen la habilidad de aprender las características de las imágenes de entrenamiento e "imaginar" nuevas imágenes propias utilizando los patrones aprendidos.

Consiste en dos redes neuronales (generador y discriminador) donde una compite contra la otra y van aprendiendo una de la otra.

- **El Generador:** intenta convertir ruido aleatorio en observaciones que parecen haber sido tomadas del conjunto de datos original.
- **El Discriminador:** trata de predecir si una observación proviene del conjunto de datos original o si es una falsificación creada por el generador.





Embeddings y Autoencoders



Embeddings (Representaciones Vectoriales)

Objetivo:

Transformar datos categóricos o palabras en vectores de menor dimensión, preservando la información semántica.

Uso Principal: Procesamiento de lenguaje natural (NLP) y modelos de clasificación. Ejemplo: Word2Vec, GloVe, BERT embeddings.

Aplicaciones:

- Representan palabras o categorías en un espacio vectorial.
- Capturan similitudes semánticas y relaciones.
- Se entrenan con grandes corpus de datos de texto para aprender asociaciones.
- Ejemplo: La palabra "gato" tendría un embedding cercano a "perro" porque tienen significados similares.



Autoencoders (Compresión y Reconstrucción)

Objetivo:

Aprender una representación comprimida de los datos para luego reconstruirlos lo mejor posible.

Uso principal:

Reducción de dimensionalidad, eliminación de ruido, generación de datos.

Ejemplo: Variational Autoencoders (VAEs) generan imágenes similares a las de entrenamiento.

Aplicaciones:

- Se dividen en Encoder (compresión) y Decoder (reconstrucción).
- Útiles para generar datos o detectar anomalías. Aplicaciones en imágenes, series temporales y texto.

Ejemplo: Comprimir imágenes grandes y luego reconstruirlas con mínima pérdida de información.



Transformers

Los transformers son un tipo de arquitectura de redes neuronales ampliamente utilizada en tareas de procesamiento de lenguaje natural (NLP), como traducción, clasificación de texto y respuesta automática a preguntas. Fueron introducidos en el influyente artículo "Attention Is All You Need" por Vaswani et al. en 2017.

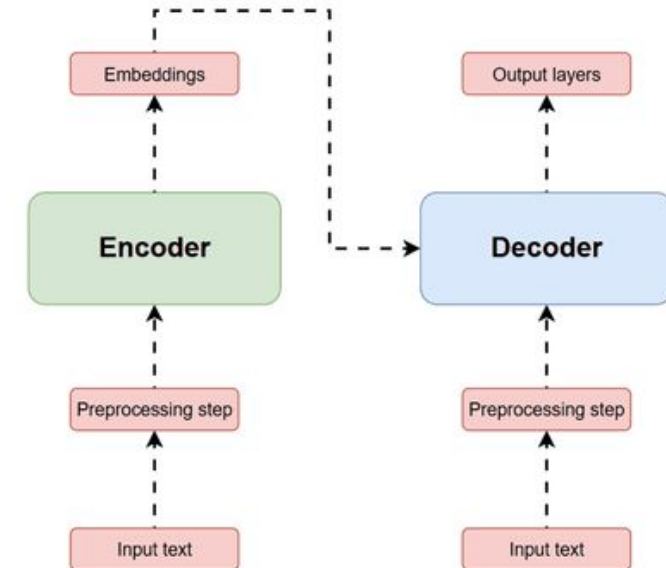
Un transformer está compuesto por **capas**. Cada capa contiene dos componentes principales:

Mecanismo de auto-atención (Self-Attention):

Permite al modelo analizar las relaciones entre distintas partes del texto de entrada. Funciona asignando un peso a cada palabra de la secuencia, indicando su relevancia dentro del contexto actual. Esto permite al modelo centrarse en las palabras importantes y reducir el peso de las menos relevantes.

Redes neuronales feed-forward:

Son perceptrones multicapa que procesan la salida del mecanismo de auto-atención. Se encargan de aprender relaciones complejas entre las palabras del texto de entrada.



Estos transformers pueden actuar solo como encoder (BERT), sólo como decoder (GPT) o como encoder-decoder (T5, BART)

Resumen de arquitecturas y usos



Modelos DL	Tareas/Usos Adecuados	Ventajas
Perceptrón	Tareas de clasificación binaria, datos linealmente separables	Simplicidad, eficiencia en términos de recursos computacionales
Perceptrón Multicapa (MLP)	Tareas de clasificación multiclase, tareas de regresión; puede manejar datos no linealmente separables	Capacidad para modelar relaciones no lineales, versatilidad en distintas tareas
Red Neuronal Convolucional (CNN)	Tareas de procesamiento de imágenes y videos, como clasificación de imágenes, detección de objetos, etc.	Capacidad para capturar características espaciales, extracción automática de características
Red Neuronal Recurrente (RNN)	Tareas con datos secuenciales, como predicción de series temporales, procesamiento del lenguaje natural, etc.	Capacidad para manejar relaciones de entrada/salida secuenciales
Memoria a Largo Plazo (LSTM)	Iguales que las RNN, pero especialmente útiles para secuencias más largas	Supera el problema del gradiente desvanecido de las RNN, mejor aprendizaje de dependencias a largo plazo
Red Neuronal Probabilística (PNN)	Tareas de clasificación, especialmente cuando el tamaño del conjunto de entrenamiento es pequeño	Ofrece una salida probabilística, es robusta al sobreajuste gracias a su simplicidad
Red Generativa Antagónica (GAN)	Generación de imágenes, síntesis de imagen a partir de texto, etc.	Capacidad para generar nuevos datos que son similares a los datos de entrenamiento
Transformer	Tareas de procesamiento del lenguaje natural, como traducción automática, resumen de textos, etc.	Captura dependencias globales en los datos, altamente paralelizables